

Sep 21, 22 18:13

main.py

Page 1/2

```
#!/bin/env python3.8

"""
Michael Bentivegna
Deep Learning Assignment 3
Professor Curro

Data from: https://www.kaggle.com/datasets/oddrational/mnist-in-csv
"""

import os
import logging
import tensorflow as tf
import pandas as pd
import sklearn.model_selection
from absl import app
from absl import flags

script_path = os.path.dirname(os.path.realpath(__file__))

# Define default values for flags
FLAGS = flags.FLAGS
flags.DEFINE_integer("batch_size", 64, "Number of samples in batch")
flags.DEFINE_float("momentum", 0.9, "Momentum of optimizer")
flags.DEFINE_integer("num_epochs", 2, "Number of SGD iterations")
flags.DEFINE_float("learning_rate", 0.02, "Learning rate / step size for SGD")
flags.DEFINE_bool("debug", False, "Set logging level to debug")

def processDataframe(df):
    """
    Helper function for getData() to put pixel information in proper form
    """
    mnist_data = df.values
    x = mnist_data[:, 1:].reshape(-1, 28, 28, 1).astype("float32") / 255
    y = mnist_data[:, 0]
    y_cat = tf.keras.utils.to_categorical(y, 10)

    return x, y_cat

def getData():
    """
    Get pixel data from MNIST dataset and split it into train, validation, and test sets
    """
    df_test = pd.read_csv(script_path + "/mnist_test.csv", delimiter=",")
    df_all_train = pd.read_csv(script_path + "/mnist_train.csv", delimiter=",")

    df_train, df_validation = sklearn.model_selection.train_test_split(
        df_all_train, test_size=(1 / 6)
    )

    x_train, y_train = processDataframe(df_train)
    x_valid, y_valid = processDataframe(df_validation)
    x_test, y_test = processDataframe(df_test)

    return x_train, y_train, x_valid, y_valid, x_test, y_test

def main(a):
    logging.basicConfig()
```

Sep 21, 22 18:13

main.py

Page 2/2

```
if FLAGS.debug:
    logging.getLogger().setLevel(logging.DEBUG)

# Get Data
x_train, y_train, x_valid, y_valid, x_test, y_test = getData()

# Set up convolutional model
model = tf.keras.models.Sequential()
model.add(
    tf.keras.layers.Conv2D(
        32,
        (3, 3),
        activation="relu",
        kernel_initializer="he_uniform",
        padding="same",
        input_shape=(28, 28, 1),
    )
)
model.add(
    tf.keras.layers.Conv2D(
        32,
        (3, 3),
        activation="relu",
        kernel_initializer="he_uniform",
        padding="same",
        input_shape=(28, 28, 1),
    )
)

# Add dense layer with dropout and l2 regularization
model.add(tf.keras.layers.MaxPooling2D(pool_size=(2, 2)))
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(10, activation="softmax", kernel_regularizer=
"l2"))
model.add(tf.keras.layers.Dropout(0.1))

# Set up optimizer with momentum
optimizer = tf.keras.optimizers.SGD(
    learning_rate=FLAGS.learning_rate, momentum=FLAGS.momentum
)
model.compile(
    optimizer=optimizer, loss="categorical_crossentropy", metrics=["accuracy"]
)

# Train model with added validation data to see accuracy on untrained data
print("-----Training-----")
model.fit(
    x_train,
    y_train,
    epochs=FLAGS.num_epochs,
    batch_size=FLAGS.batch_size,
    validation_data=(x_valid, y_valid),
    verbose=1,
)

# Check on test data (only ran at the end)
print("-----Test Accuracy-----")
model.evaluate(x_test, y_test, verbose=1)

if __name__ == "__main__":
    app.run(main)
```